Image Processing

Lab Manual

Himanshu Sardana 102303244

Table of Contents

Assignment 1

Question 1

Read an image using the `imread()` function.

Solution

```
import cv2
img = cv2.imread(r"test.jpg")
print(img)
```

Output

```
[[[ 76  44   3]
  [ 68  37   0]
  [ 66  34   0]
  ...
  [  6  85  66]
  [  0  46  29]
  [ 17  60  45]]

 [[ 79  46   1]
  [ 73  41   0]
  [ 70  39   0]
  ...
  [ 60 141 122]
  [ 46 106  88]
  [ 60 109  93]]

 [[ 89  56   6]
  [ 87  54   5]
  [ 83  53   6]
  ...
  [ 84 173 153]
  [ 56 127 110]
  [ 64 127 111]]

 ...

 [[  3  46   3]
  [  0  34   0]
  [  0  37   0]
  ...
  [ 14  18  12]
  [  0   5   0]
  [ 19  27  17]]

 [[ 11  47  11]
  [  0  33   0]
  [  0  34   0]
  ...
  [ 15  19  13]
  [ 17  25  15]
  [ 17  25  15]]

 [[ 21  54  20]
  [  4  37   3]
```

```
[  5  37   6]
...
[ 78  82  76]
[ 89  97  87]
[ 44  52  42]]]
```

QUESTION 2

Get the height and width of the image.

SOLUTION

```
img = cv2.imread(r"test.jpg")

print(f"Height: {img.shape[0]}  Width: {img.shape[1]}")
```

OUTPUT

```
Height: 1000  Width: 1600
```

QUESTION 3

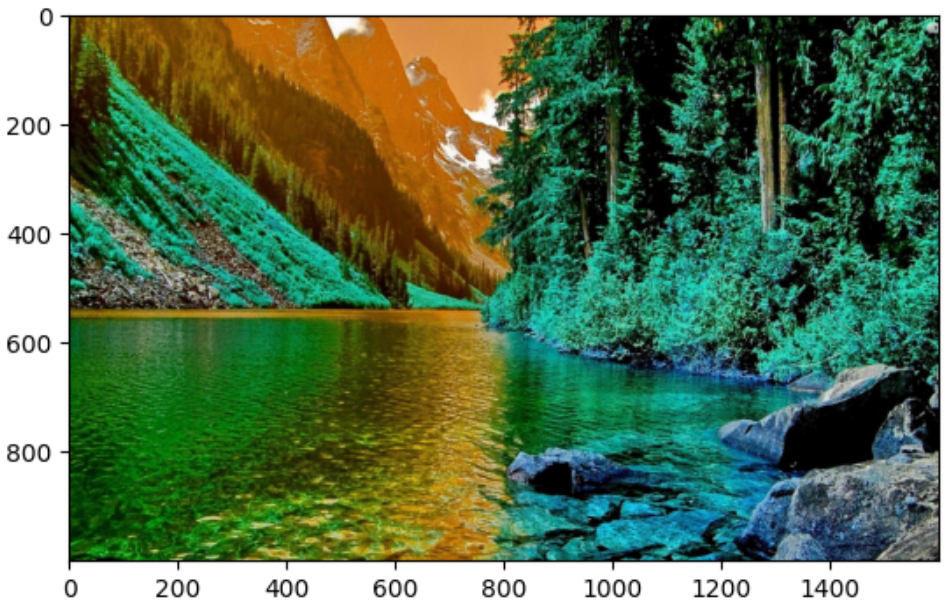
Display the image

SOLUTION

```
from matplotlib import pyplot as plt
plt.imshow(img)
```

OUTPUT

```
<matplotlib.image.AxesImage at 0x7fd02a2a4b90><Figure size 640x480 with 1 Axes>
```



Question 4

Extract the RGB values of the image.

Solution

```
B, G, R = img[100][100]
print(f"Red: {R}\nGreen: {G}\nBlue: {B}")
```

Output

```
Red: 4
Green: 38
Blue: 75
```

Question 5

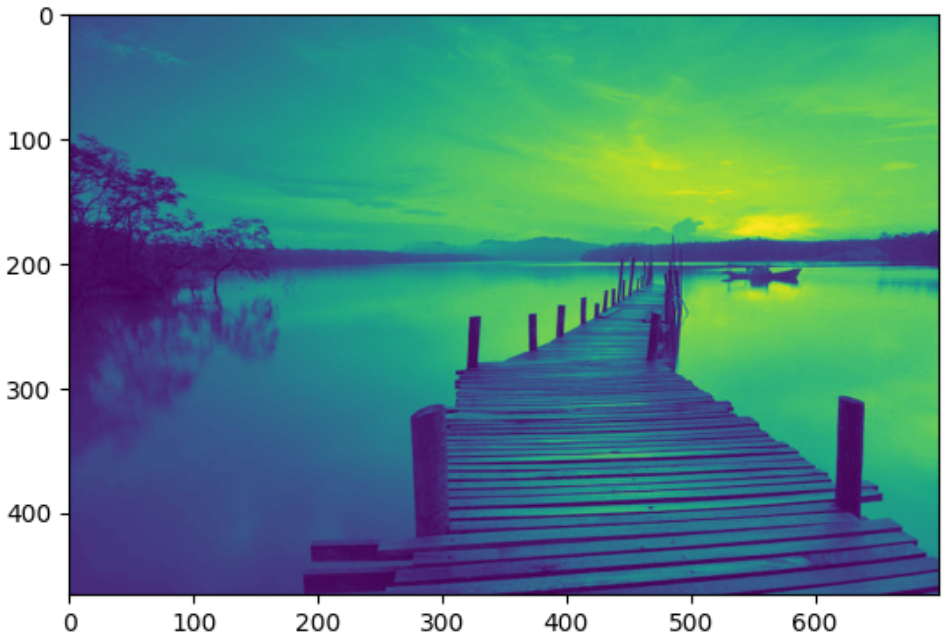
Convert an image to grayscale

Solution

```
img = cv2.imread("gray_original.png")

gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.imshow(gray_img)
cv2.imwrite("gray_greyscale.png", gray_img)
```

Output

```
True<Figure size 640x480 with 1 Axes>
```



Question 6

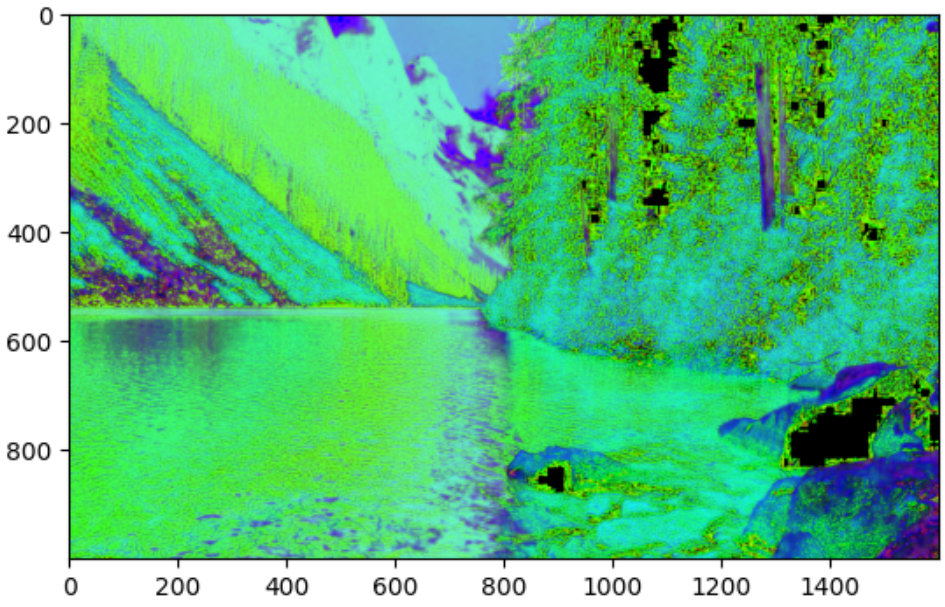
Comvert a BGR image to HSV

Solution

```
img = cv2.imread("test.jpg")

hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
plt.imshow(hsv_img)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd024f2da50><Figure size 640x480 with 1 Axes>
```



Question 7

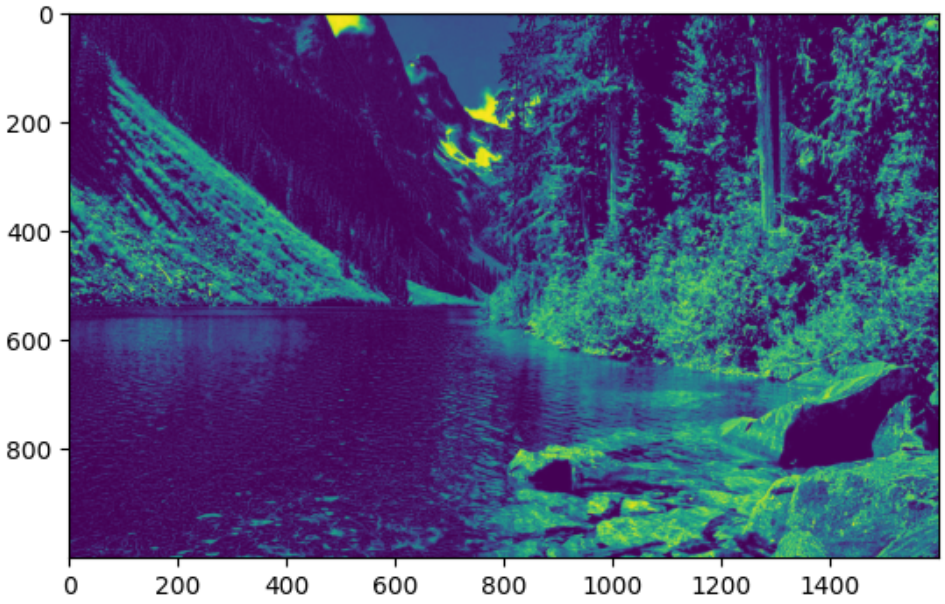
Split the R, G and B channels of an image

Solution

```
img = cv2.imread("test.jpg")

B, G, R = cv2.split(img)
plt.imshow(B)
plt.imshow(G)
plt.imshow(R)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd02a0f3890><Figure size 640x480 with 1 Axes>
```


0
200
400
600
800
0
200
400
600
800
1000
1200
1400

Assignment 2

Question 1

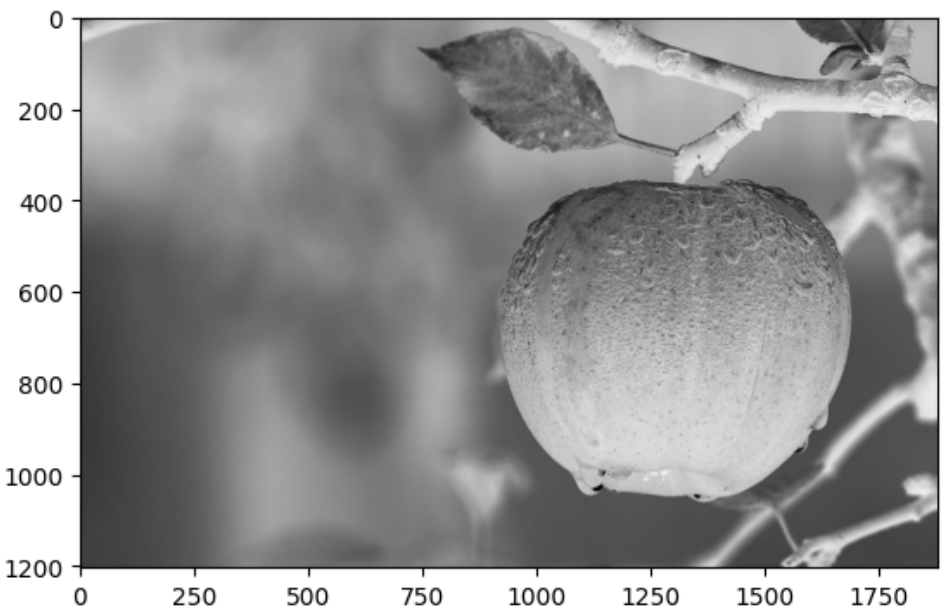
Write a program to find the complement of an image

Solution

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("./gray.jpeg")
# print(img)

complement = 255 - img
plt.imshow(complement)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd02a05da10><Figure size 640x480 with 1 Axes>
```



Question 2

Convert image to grayscale using

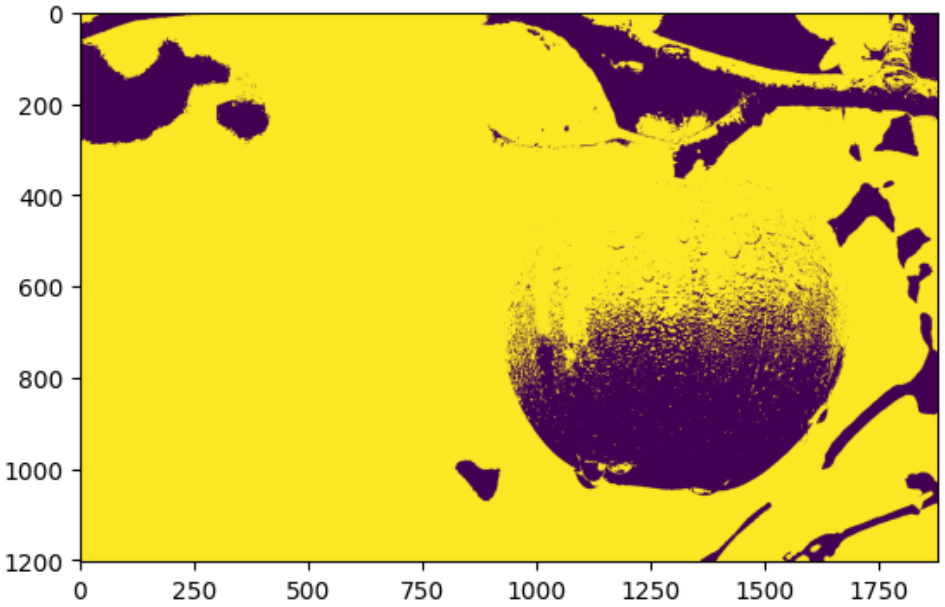
Solution

```
import cv2
import numpy as np

img = cv2.imread('./gray.jpeg', cv2.IMREAD_GRAYSCALE)

# Using mean threshold
```

```
_, binary_img = cv2.threshold(img, np.mean(img), 255, cv2.THRESH_BINARY)
plt.imshow(binary_img)

# Usin user-defined threshold
threshold = 80
_, binary_img = cv2.threshold(img, threshold, 255, cv2.THRESH_BINARY)
plt.imshow(binary_img)
```

OUTPUT

```
<matplotlib.image.AxesImage at 0x7fd024f38310><Figure size 640x480 with 1 Axes>
```



QUESTION 3

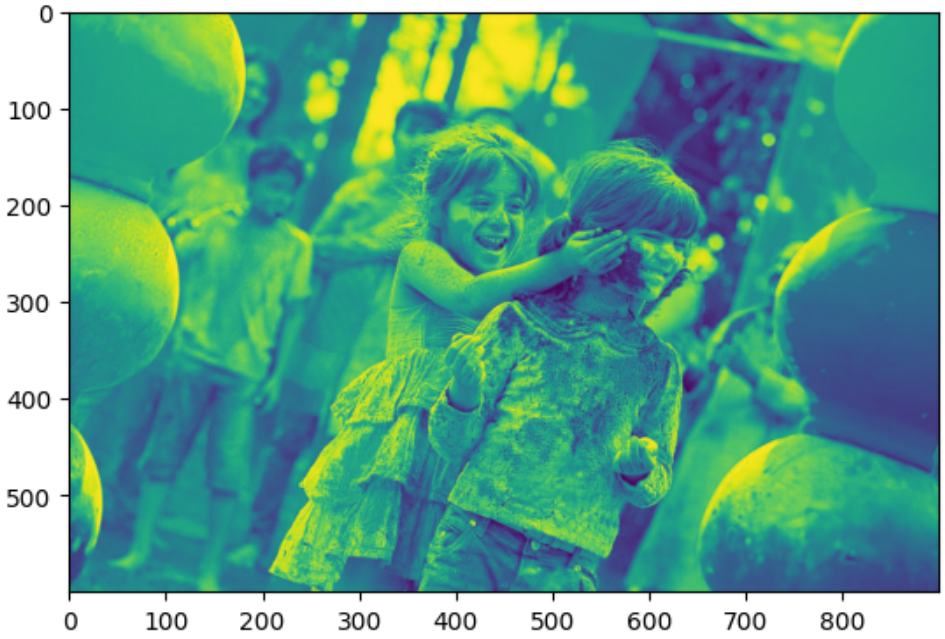
Convert a colored image to a grayscale image by: (a) taking mean average of all 3 planes (b) User-defined weightage

SOLUTION

```
img = cv2.imread('./color.jpg')

grayscale = np.mean(img, axis=2)
# convert grayscale to unsigned integers (get rid of decimals/floating point numbers)
grayscale.astype(np.uint8)
plt.imshow(grayscale)

weightage_r = 0.6
weightage_g = 0.0
weightage_b = 0.4

img = cv2.imread('./holi.jpg')
gray_img = (weightage_r * img[:, :, 2]) + (weightage_g * img[:, :, 1]) + (weightage_b *
img[:, :, 0])

plt.imshow(gray_img)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd02482b890><Figure size 640x480 with 1 Axes>
```



Question 4

Draw a border around an image.

Solution

```
img = cv2.imread('./gray.jpeg', cv2.IMREAD_GRAYSCALE)
border_width = 200

img_with_border = cv2.copyMakeBorder(img, border_width, border_width, border_width,
border_width, cv2.BORDER_CONSTANT, value=(255, 0, 0))
plt.imshow(img_with_border)
```

Assignment 3

Question 1

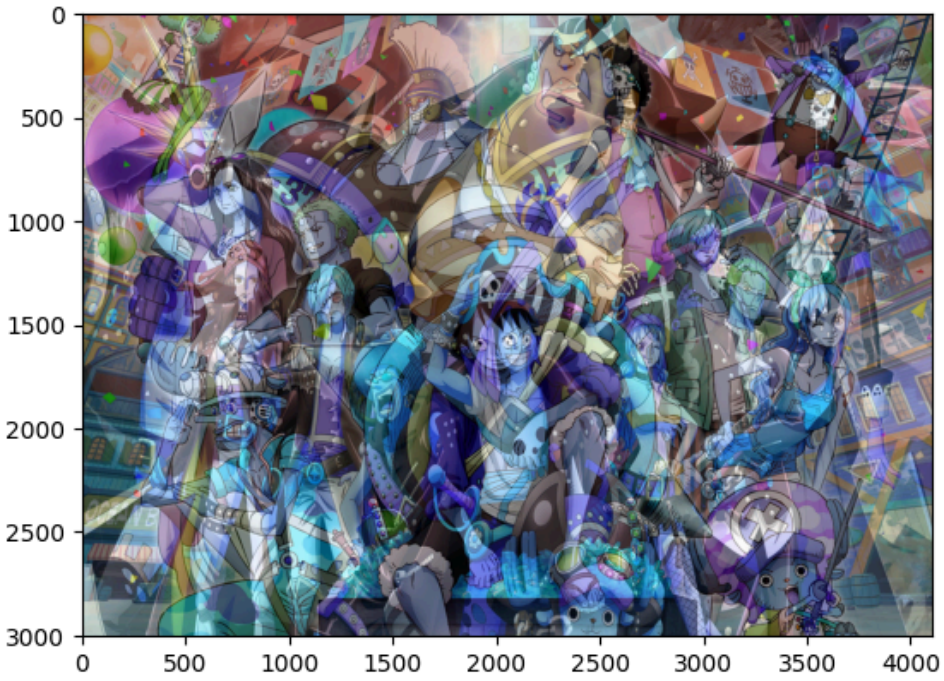
Add 2 images

Solution

```
import matplotlib.pyplot as plt
import cv2
import numpy as np

img1 = cv2.imread('./img1.jpg')
img2 = cv2.imread('./img2.jpg')

img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

alpha = 0.5
beta = 1 - alpha
img = cv2.addWeighted(img1, alpha, img2, beta, 0)

cv2.imwrite('blended_image.jpg', img)
# cv2.imshow('Blended Image', img)
plt.imshow(img)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd02489da50><Figure size 640x480 with 1 Axes>
```



Question 2

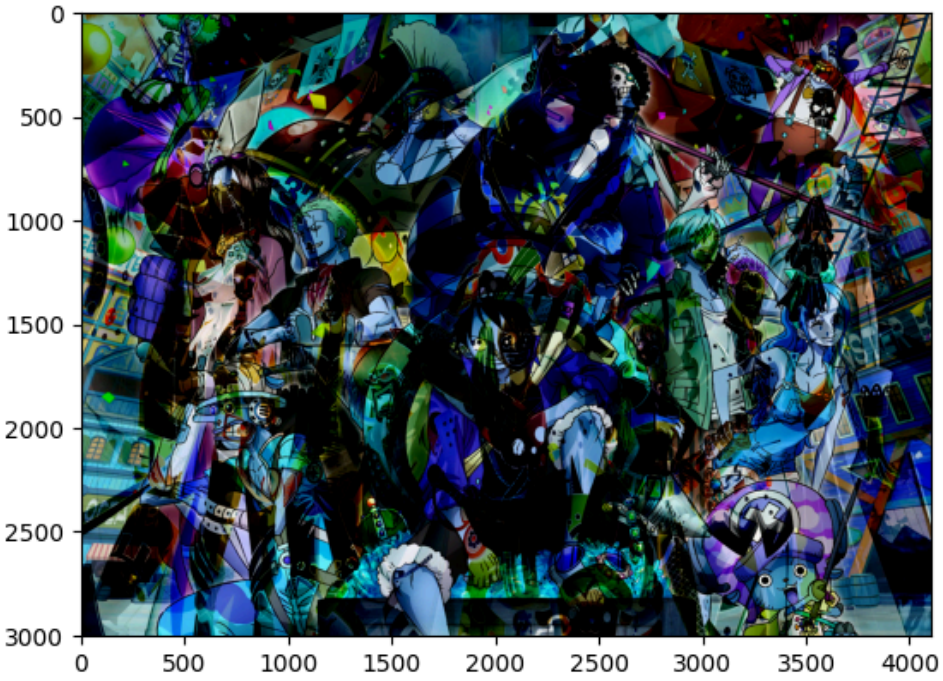
Subtract 2 images

Solution

```
import matplotlib.pyplot as plt
import cv2
import numpy as np

img1 = cv2.imread('./img1.jpg')
img2 = cv2.imread('./img2.jpg')

img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

img = cv2.subtract(img1, img2)

cv2.imwrite('blended_image.jpg', img)
# cv2.imshow('Blended Image', img)
plt.imshow(img)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd0248f5a50><Figure size 640x480 with 1 Axes>
```



Question 3

Multiply 2 images

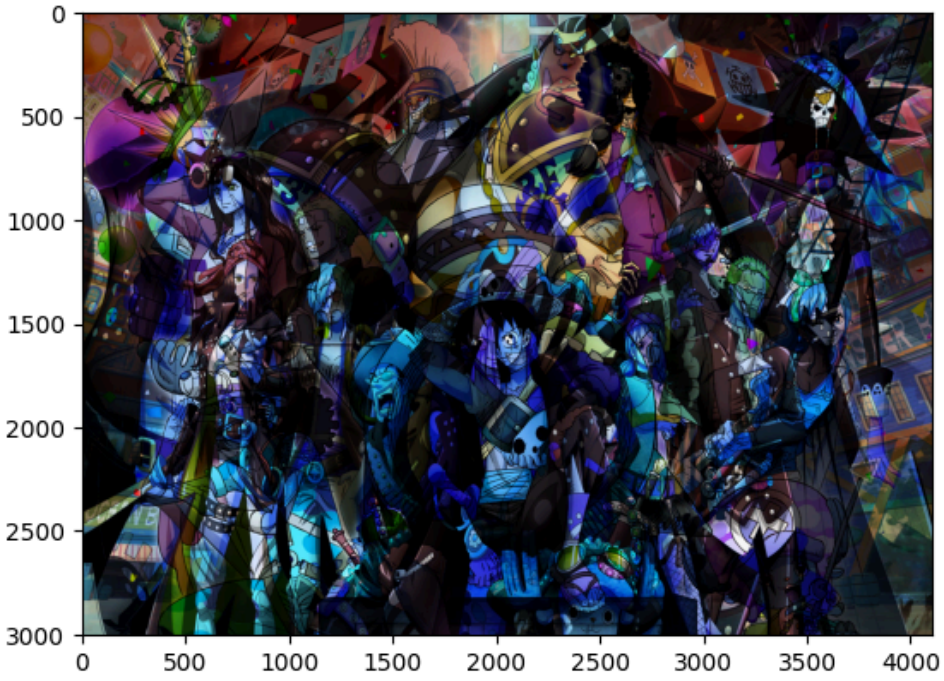
Solution

```
import matplotlib.pyplot as plt
import cv2
import numpy as np

img1 = cv2.imread('./img1.jpg')
img2 = cv2.imread('./img2.jpg')
```

```
img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

img = cv2.multiply(img1, img2, scale=1.0/255.0)

cv2.imwrite('blended_image.jpg', img)
# cv2.imshow('Blended Image', img)
plt.imshow(img)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd02490da50><Figure size 640x480 with 1 Axes>
```



Question 4

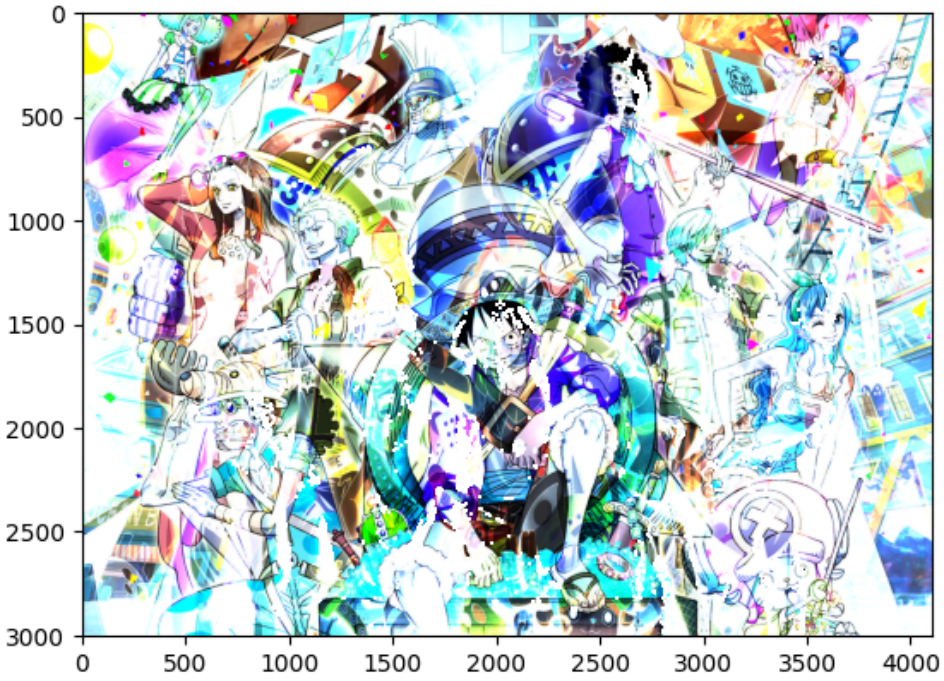
Divide 2 images

Solution

```
import matplotlib.pyplot as plt
import cv2
import numpy as np

img1 = cv2.imread('./img1.jpg')
img2 = cv2.imread('./img2.jpg')

img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

img = np.array(img1, dtype=np.float32)/np.array(img2, dtype=np.float32)

cv2.imwrite('blended_image.jpg', img)
# cv2.imshow('Blended Image', img)
plt.imshow(img)
```

Output

```
/tmp/ipykernel_2436/2660822262.py:10: RuntimeWarning: divide by zero encountered in divide
  img = np.array(img1, dtype=np.float32)/np.array(img2, dtype=np.float32)
/tmp/ipykernel_2436/2660822262.py:10: RuntimeWarning: invalid value encountered in divide
  img = np.array(img1, dtype=np.float32)/np.array(img2, dtype=np.float32)
[ WARN:0@68.598] global loadsave.cpp:848 imwrite_ Unsupported depth image for selected
encoder is fallbacked to CV_8U.
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or
[0..255] for integers). Got range [0.0..255.0].
<matplotlib.image.AxesImage at 0x7fd02462a010><Figure size 640x480 with 1 Axes>
```



Question 5

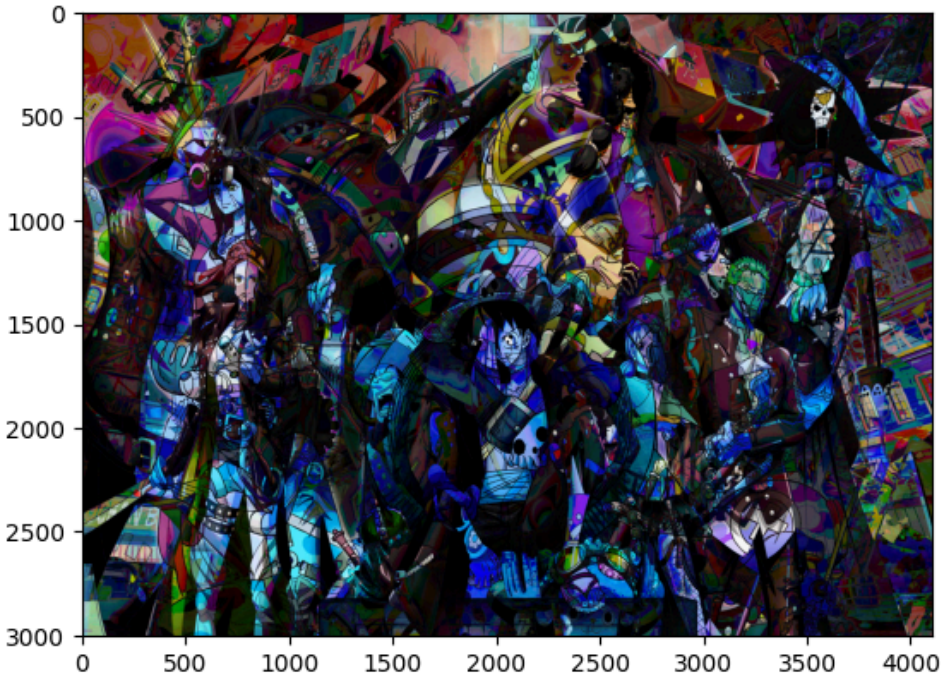
AND 2 images

Solution

```python
import matplotlib.pyplot as plt
import cv2
import numpy as np

img1 = cv2.imread('./img1.jpg')
img2 = cv2.imread('./img2.jpg')

img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

img = np.bitwise_and(np.array(img1, dtype=np.uint8), np.array(img2, dtype=np.uint8))

cv2.imwrite('blended_image.jpg', img)
# cv2.imshow('Blended Image', img)
plt.imshow(img)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd02469aa10><Figure size 640x480 with 1 Axes>
```



Question 6

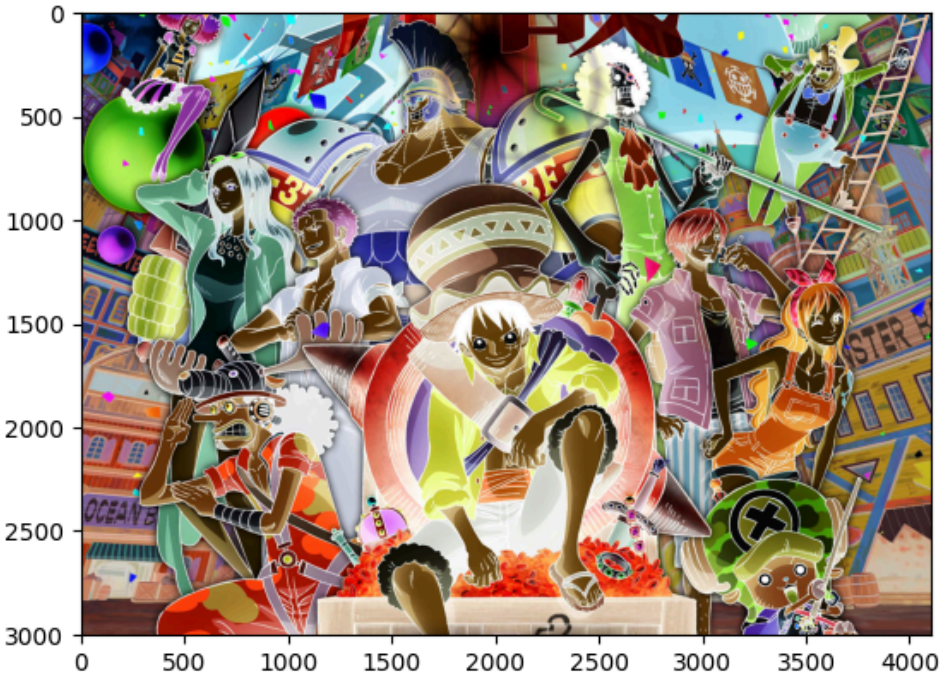
NOT 2 images

Solution

```python
import matplotlib.pyplot as plt
import cv2
import numpy as np

img1 = cv2.imread('./img1.jpg')
img2 = cv2.imread('./img2.jpg')

img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

img = np.bitwise_not(np.array(img1, dtype=np.uint8), np.array(img2, dtype=np.uint8))

cv2.imwrite('blended_image.jpg', img)
# cv2.imshow('Blended Image', img)
plt.imshow(img)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd024513f90><Figure size 640x480 with 1 Axes>
```



Question 7

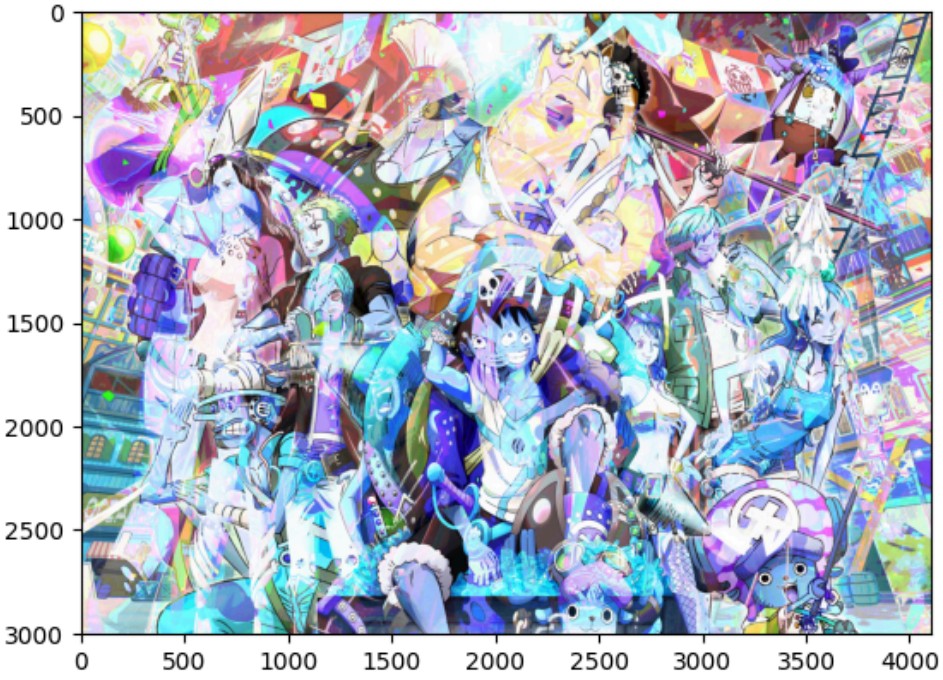
OR 2 images

Solution

```
import matplotlib.pyplot as plt
import cv2
import numpy as np

img1 = cv2.imread('./img1.jpg')
img2 = cv2.imread('./img2.jpg')

img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

img = np.bitwise_or(np.array(img1, dtype=np.uint8), np.array(img2, dtype=np.uint8))

cv2.imwrite('blended_image.jpg', img)
# cv2.imshow('Blended Image', img)
plt.imshow(img)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd024589610><Figure size 640x480 with 1 Axes>
```


0
500
1000
1500
2000
2500
3000
0
500
1000
1500
2000
2500
3000
3500
4000

Assignment 4

Question 1

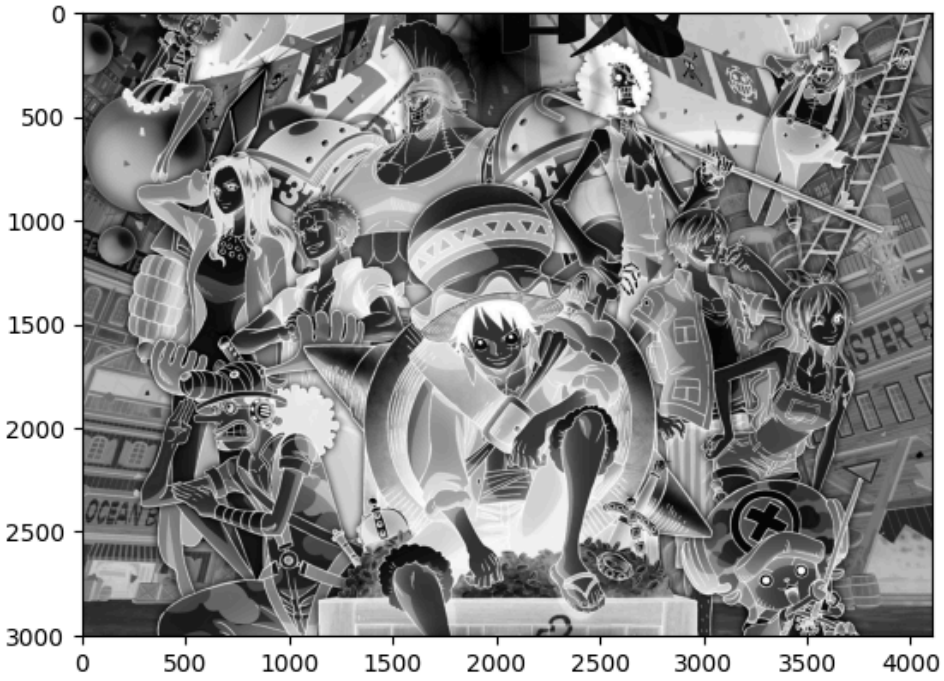
Apply the negative image transformation

Solution

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("gray_image.jpg")

# Apply negative transformation
negative_img = np.max(img) - img
print(np.max(img))

cv2.imwrite("negative.jpg", negative_img)
plt.imshow(negative_img)
```

Output

```
255
<matplotlib.image.AxesImage at 0x7fd0245a5a50><Figure size 640x480 with 1 Axes>
```



Question 2

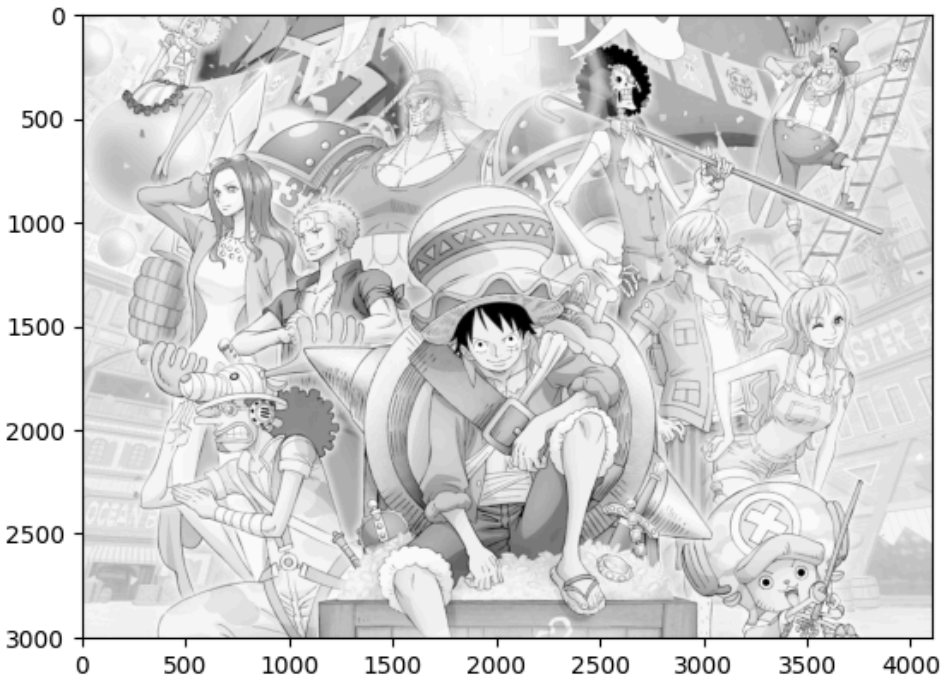
Apply the logarithmic image transformation

Solution

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("gray_image.jpg")

img = img.astype(np.float32)

c = 255 / np.log(1 + np.max(img))
log_img = np.uint8(c * np.log1p(img))

cv2.imwrite("log_img.jpg", log_img)
plt.imshow(log_img)
```

Output

```
<matplotlib.image.AxesImage at 0x7fd024476310><Figure size 640x480 with 1 Axes>
```



Question 3

Make a custom 3x3 image and repeatedly apply the log transform

Solution

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

new_img = (np.random.rand(3, 3) * 255).astype(np.uint8)


fig, axes = plt.subplots(1, 11, figsize=(2.2*(11), 4))
```

```
axes[0].imshow(new_img)
axes[0].set_title("Original")
axes[0].axis("off")

for i in range(10):
    c = 255 / np.log(1 + float(np.max(new_img)))
    log_img = np.uint8(c * np.log1p(new_img))

    axes[i+1].imshow(log_img)
    axes[i+1].set_title(f"Transformation {i+1}")
    axes[i+1].axis("off")

    new_img = log_img

plt.tight_layout()
plt.show()
```

OUTPUT

```
<Figure size 2420x400 with 11 Axes>
```

QUESTION 3

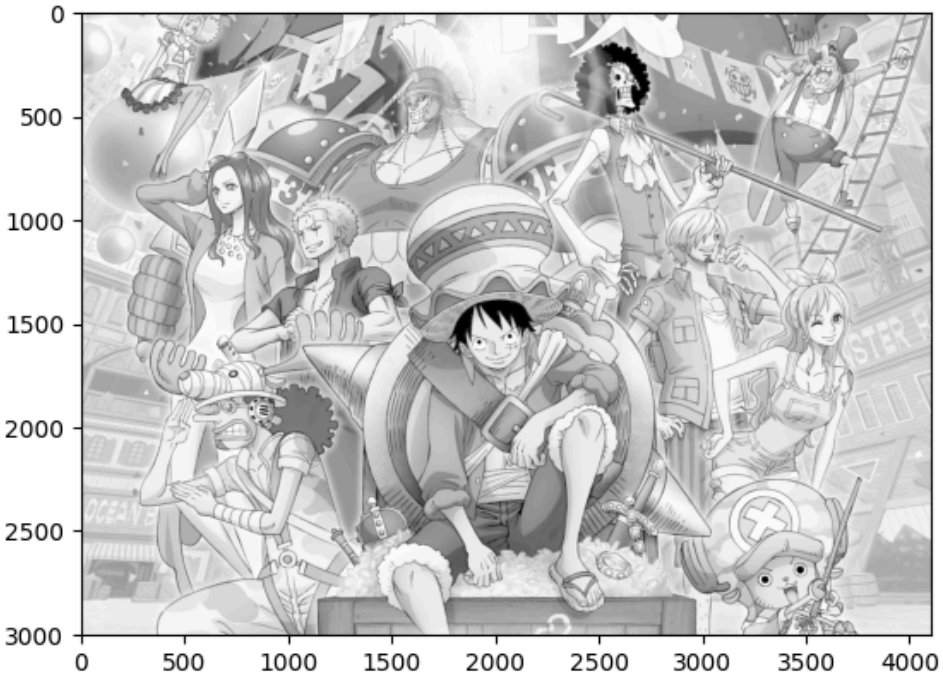
Apply the Power Law (Gamma Correction) image transformation

SOLUTION

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("gray_image.jpg")

gamma = 0.3
c = 1.0

norm_img = img / 255

power_log_img = c * np.power(norm_img, gamma)
plt.imshow(power_log_img)
```

OUTPUT

```
<matplotlib.image.AxesImage at 0x7fd01df0be10><Figure size 640x480 with 1 Axes>
```



Question 5

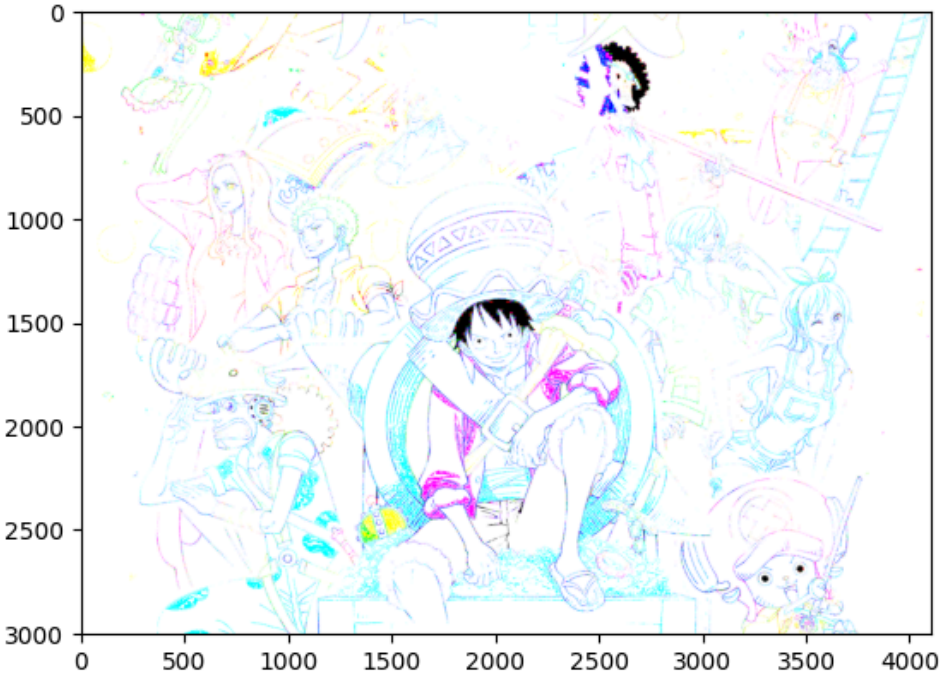
Apply the Contrast Stretching Transformation

Solution

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("img1.jpg").astype(np.float32)

r1, r2 = 80, 120
s1, s2 = 0, 100

mask = (img >= r1) & (img <= r2)

img[mask] = ((img[mask] - 1) / (r2 - r1)) * (s2 - s1) + s1
img = np.clip(img, 0, 255).astype(np.float32)
plt.imshow(img)
```

Output

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or
[0..255] for integers). Got range [0.0..255.0].
<matplotlib.image.AxesImage at 0x7fd01df6da50><Figure size 640x480 with 1 Axes>
```


0
500
1000
1500
2000
2500
3000
0
500
1000
1500
2000
2500
3000
3500
4000

Assignment 5

Question 1

Build a Convolutional Neural Network to predict digits using the MNIST datasets

Solution

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models

(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

x_train = x_train.reshape((x_train.shape[0], 28, 28, 1))
x_test = x_test.reshape((x_test.shape[0], 28, 28, 1))

x_train, x_test = x_train / 255.0, x_test / 255.0

model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(
    x_train, y_train, epochs=5, batch_size=64, validation_split=0.1
)

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print('\nTest accuracy:', test_acc)
```

Output

```
2025-10-28 19:39:53.892312: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not
find cuda drivers on your machine, GPU will not be used.
2025-10-28 19:39:54.415760: I tensorflow/core/platform/cpu_feature_guard.cc:210] This
TensorFlow binary is optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with
the appropriate compiler flags.
2025-10-28 19:39:56.498143: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not
find cuda drivers on your machine, GPU will not be used.
/home/himanshu/assignments/.venv/lib/python3.11/site-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as
the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2025-10-28 19:39:58.661078: E external/local_xla/xla/stream_executor/cuda/
```

```
cuda_platform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit:
UNKNOWN ERROR (303)
Epoch 1/5
2025-10-28 19:39:59.060312: W external/local_xla/xla/tsl/framework/cpu_allocator_impl.cc:84]
Allocation of 169344000 exceeds 10% of free system memory.
□[1m844/844□[0m □[32m━━━━━━━━━━━━━━━━━━━━□[0m□[37m□[0m □[1m22s□[0m 24ms/step - accuracy:
0.9406 - loss: 0.2002 - val_accuracy: 0.9800 - val_loss: 0.0703
Epoch 2/5
□[1m844/844□[0m □[32m━━━━━━━━━━━━━━━━━━━━□[0m□[37m□[0m □[1m20s□[0m 24ms/step - accuracy:
0.9814 - loss: 0.0613 - val_accuracy: 0.9852 - val_loss: 0.0524
Epoch 3/5
□[1m844/844□[0m □[32m━━━━━━━━━━━━━━━━━━━━□[0m□[37m□[0m □[1m20s□[0m 23ms/step - accuracy:
0.9866 - loss: 0.0434 - val_accuracy: 0.9868 - val_loss: 0.0457
Epoch 4/5
□[1m844/844□[0m □[32m━━━━━━━━━━━━━━━━━━━━□[0m□[37m□[0m □[1m20s□[0m 24ms/step - accuracy:
0.9889 - loss: 0.0338 - val_accuracy: 0.9900 - val_loss: 0.0408
Epoch 5/5
□[1m844/844□[0m □[32m━━━━━━━━━━━━━━━━━━━━□[0m□[37m□[0m □[1m20s□[0m 23ms/step - accuracy:
0.9911 - loss: 0.0269 - val_accuracy: 0.9887 - val_loss: 0.0403
2025-10-28 19:41:40.712464: W external/local_xla/xla/tsl/framework/cpu_allocator_impl.cc:84]
Allocation of 31360000 exceeds 10% of free system memory.
313/313 - 2s - 5ms/step - accuracy: 0.9898 - loss: 0.0315

Test accuracy: 0.989799976348877
```

Assignment 6

Question 1

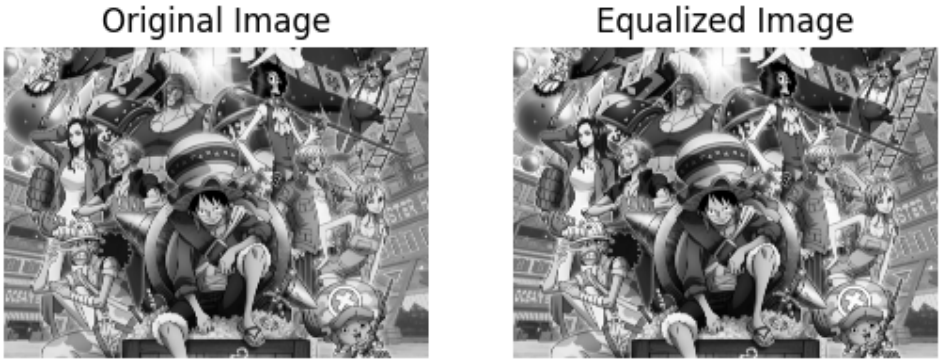
WAP to enhance the input image using histogram equalization.

Solution

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("img1.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
final_img = cv2.equalizeHist(gray)

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(final_img, cmap='gray')
plt.title('Equalized Image')
plt.axis('off')
```

Output

```
(np.float64(-0.5), np.float64(4102.5), np.float64(3003.5), np.float64(-0.5))<Figure size
640x480 with 2 Axes>
```



Question 2

WAP to match the histogram of the input image with that of reference image using histogram matching technique.

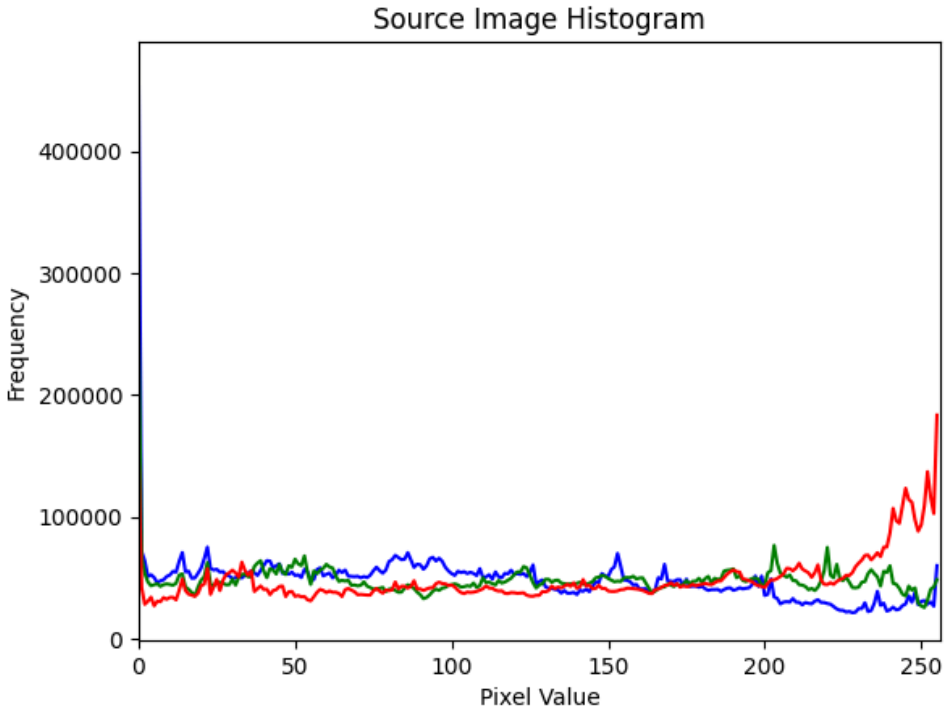
Solution

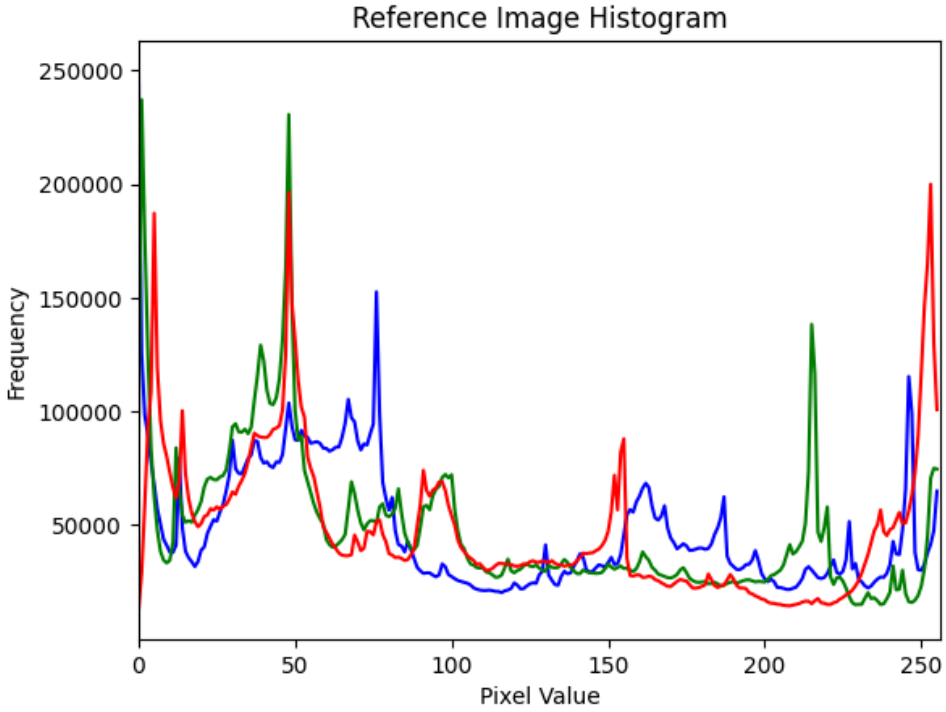
```
import numpy as np
import cv2
import matplotlib.pyplot as plt
```

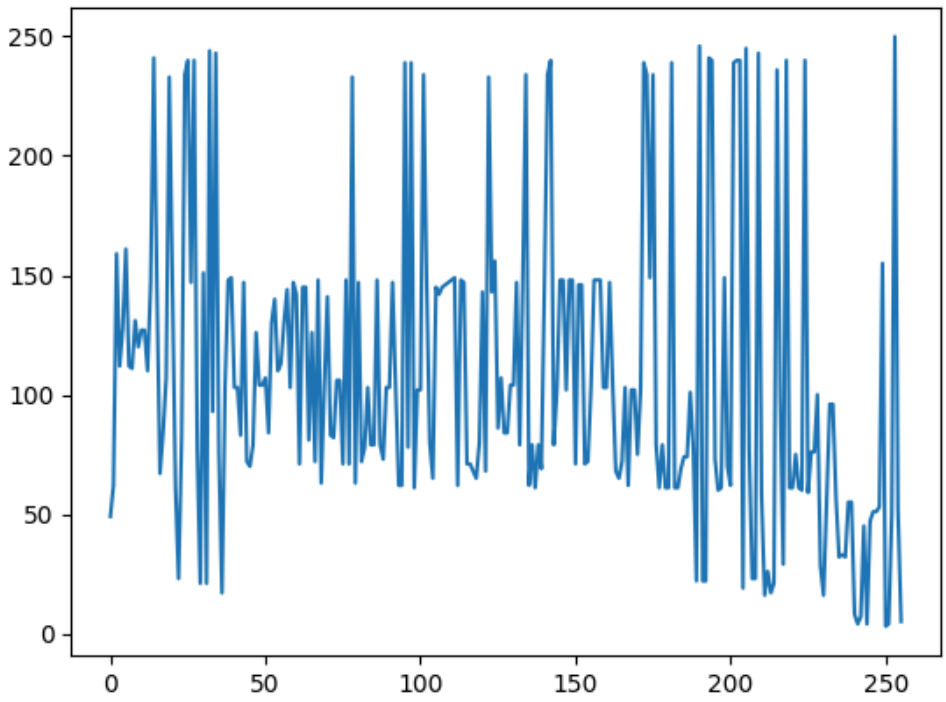
```
src = cv2.imread("img1.jpg")
ref = cv2.imread("img2.jpg")

ref = cv2.resize(ref, (src.shape[1], src.shape[0]))

def display_histogram(image, title):
    color = ('b', 'g', 'r')
    plt.figure()
    plt.title(title)
    plt.xlabel('Pixel Value')
    plt.ylabel('Frequency')
    values = []
    for i, col in enumerate(color):
        hist = cv2.calcHist([image], [i], None, [256], [0, 256])
        values = hist.copy()
        plt.plot(hist, color=col)
        plt.xlim([0, 256])
    plt.show()
    return values

img1_vals = display_histogram(src, 'Source Image Histogram').flatten()
img2_vals = display_histogram(ref, 'Reference Image Histogram').flatten()

matched_vals = img1_vals.copy()
color = ('b', 'g', 'r')
for i in range(256):
    diff = np.abs(img1_vals[i] - img2_vals)
    matched_vals[i] = np.argmin(diff)
matched_vals = matched_vals.astype(np.uint8)
print(matched_vals)

plt.plot(matched_vals)
```

Output

```
<Figure size 640x480 with 1 Axes><Figure size 640x480 with 1 Axes>[ 49  62 159 112 128 161
112 111 131 120 127 127 110 147 241 147  67  84
 107 233 149  61  23  83 234 240 147 240  77  21 151  21 244  93 243  76
  17 104 148 149 103 103  83 147  72  70  79 126 104 104 107  84 130 140
 110 113 130 144 103 147 142  71 145 145  81 126  72 148  63 104 141  83
  82 106 106  71 148  71 233  63 147  72  78 103  79  79 148  79  73 103
 103 147 103  62  62 239  78 239  61 102 102 234 147  80  65 145 142 145
 146 147 148 149  62 148 147  71  71  68  65  80 143  68 233 143 156  86
 107  84  84 104 104 147  79 148 234  62  79  61  79  69 148 234 240  79
 102 148 148 102 148 148  71 146 146  71  72 103 148 148 148 103 103 147
 104  68  65  72 103  62 102 102  75 102 239 234 149 234  79  61  79  61
  61 239  61  61  69  74  74 101  77  22 246  22  22 241 240  73  60  61
 149  70  62 239 240 240  19 245  77  23  23 243  58  16  26  17  21 236
 100  29 240  61  61  75  61  60 240  59  76  76 100  28  16  57  96  96
  57  32  33  32  55  55   8   4   7  45   4  47  51  51  53 155   3   4
  50 250  52   5]
[<matplotlib.lines.Line2D at 0x7fcf3dda7c90>]<Figure size 640x480 with 1 Axes>
```


Source Image Histogram
Frequency
400000
300000
200000
100000
0
0
50
100
150
200
250
Pixel Value


Reference Image Histogram
Frequency
250000
200000
150000
100000
50000
0
50
100
150
200
250
Pixel Value



QUESTION 3

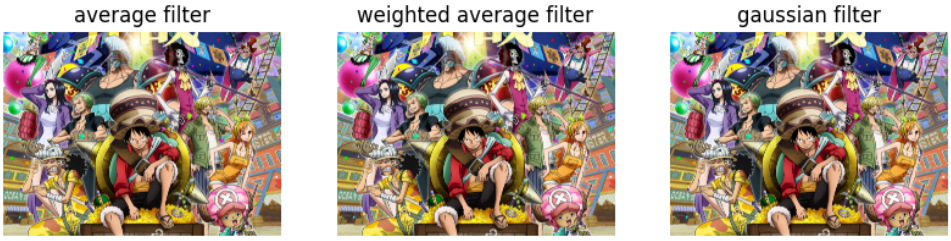
WAP to smooth the input image using:

SOLUTION

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

src = cv2.imread("img1.jpg")

# average filter
dst1 = cv2.blur(src, (5, 5))

# weighted average filter
dst2 = cv2.boxFilter(src, -1, (5, 5), normalize=True)

# gaussian filter
dst3 = cv2.GaussianBlur(src, (5, 5), 0)

plt.figure(figsize=(10, 7))
plt.subplot(1, 3, 1)
plt.title("average filter")
plt.imshow(cv2.cvtColor(dst1, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.subplot(1, 3, 2)
plt.title("weighted average filter")
plt.imshow(cv2.cvtColor(dst2, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.subplot(1, 3, 3)
plt.title("gaussian filter")
plt.imshow(cv2.cvtColor(dst3, cv2.COLOR_BGR2RGB))
plt.axis("off")
```

Output

```
(np.float64(-0.5), np.float64(4102.5), np.float64(3003.5), np.float64(-0.5))<Figure size 1000x700 with 3 Axes>
```



Assignment 7

Question 1

Write a program to generate following noises using equations derived from PDFs of noise distributions. Compare your output with those generated using inbuilt functions. Plot the histograms of the generated noise to determine the shape of the distribution.

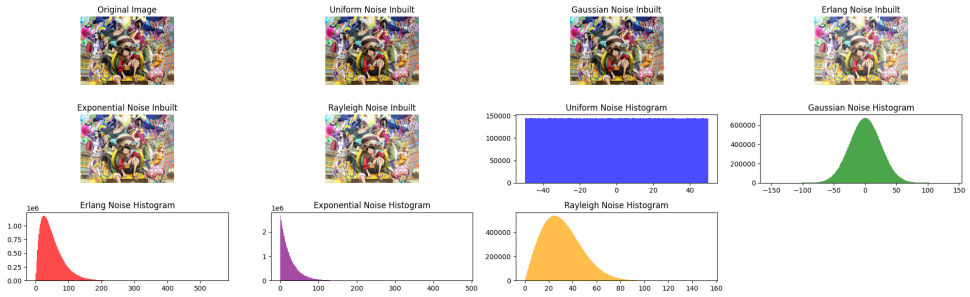
Solution

```
import numpy as np
import matplotlib.pyplot as plt
import cv2


def uniform_noise(image, a=-50, b=50):
    noise = np.random.uniform(a, b, image.shape).astype(np.float32)
    noisy_image = cv2.add(image.astype(np.float32), noise)
    return (np.clip(noisy_image, 0, 255).astype(np.uint8), noise)


def gaussian_noise(image, mean=0, sigma=25):
    noise = np.random.normal(mean, sigma, image.shape).astype(np.float32)
    noisy_image = cv2.add(image.astype(np.float32), noise)
    return (np.clip(noisy_image, 0, 255).astype(np.uint8), noise)


def erlang_noise(image, shape=2, scale=25):
    noise = np.random.gamma(shape, scale, image.shape).astype(np.float32)
    noisy_image = cv2.add(image.astype(np.float32), noise)
    return (np.clip(noisy_image, 0, 255).astype(np.uint8), noise)


def exponential_noise(image, scale=25):
    noise = np.random.exponential(scale, image.shape).astype(np.float32)
    noisy_image = cv2.add(image.astype(np.float32), noise)
    return (np.clip(noisy_image, 0, 255).astype(np.uint8), noise)


def rayleigh_noise(image, scale=25):
    noise = np.random.rayleigh(scale, image.shape).astype(np.float32)
    noisy_image = cv2.add(image.astype(np.float32), noise)
    return (np.clip(noisy_image, 0, 255).astype(np.uint8), noise)


img = cv2.imread("./img1.jpg")

plt.figure(figsize=(20, 10))
plt.subplot(5, 4, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

noisy_img = uniform_noise(img)
plt.subplot(5, 4, 2)
plt.imshow(cv2.cvtColor(noisy_img[0], cv2.COLOR_BGR2RGB))
```

```
plt.title("Uniform Noise Inbuilt")
plt.axis("off")

gauss_noisy_img = gaussian_noise(img)
plt.subplot(5, 4, 3)
plt.imshow(cv2.cvtColor(gauss_noisy_img[0], cv2.COLOR_BGR2RGB))
plt.title("Gaussian Noise Inbuilt")
plt.axis("off")

erlang_noisy_img = erlang_noise(img)
plt.subplot(5, 4, 4)
plt.imshow(cv2.cvtColor(erlang_noisy_img[0], cv2.COLOR_BGR2RGB))
plt.title("Erlang Noise Inbuilt")
plt.axis("off")

exp_noisy_img = exponential_noise(img)
plt.subplot(5, 4, 5)
plt.imshow(cv2.cvtColor(exp_noisy_img[0], cv2.COLOR_BGR2RGB))
plt.title("Exponential Noise Inbuilt")
plt.axis("off")

rayleigh_noisy_img = rayleigh_noise(img)
plt.subplot(5, 4, 6)
plt.imshow(cv2.cvtColor(rayleigh_noisy_img[0], cv2.COLOR_BGR2RGB))
plt.title("Rayleigh Noise Inbuilt")
plt.axis("off")

plt.subplot(5, 4, 7)
plt.hist(noisy_img[1].ravel(), bins=256, color="blue", alpha=0.7)
plt.title("Uniform Noise Histogram")

plt.subplot(5, 4, 8)
plt.hist(gauss_noisy_img[1].ravel(), bins=256, color="green", alpha=0.7)
plt.title("Gaussian Noise Histogram")

plt.subplot(5, 4, 9)
plt.hist(erlang_noisy_img[1].ravel(), bins=256, color="red", alpha=0.7)
plt.title("Erlang Noise Histogram")

plt.subplot(5, 4, 10)
plt.hist(exp_noisy_img[1].ravel(), bins=256, color="purple", alpha=0.7)
plt.title("Exponential Noise Histogram")

plt.subplot(5, 4, 11)
plt.hist(rayleigh_noisy_img[1].ravel(), bins=256, color="orange", alpha=0.7)
plt.title("Rayleigh Noise Histogram")

plt.tight_layout()

plt.show()
```

Output

```
<Figure size 2000x1000 with 11 Axes>
```



QUESTION 2

Write a program to use random values in confusion matrix and compute the different quantitative metrics (Accuracy, precision, recall, F1-score, MCC) for 2x2 and 3x3 matrices.

SOLUTION

```
import numpy as np

matrix_2x2 = np.random.randint(0, 100, size=(2, 2))
matrix_3x3 = np.random.randint(0, 100, size=(3, 3))

metrics = ["Accuracy", "Precision", "Recall", "F1-Score", "MCC"]

print(matrix_2x2)
print("For 2x2 Matrix")
TP = matrix_2x2[0][0]
TN = matrix_2x2[1][1]
FP = matrix_2x2[1][0]
FN = matrix_2x2[0][1]

accuracy = (TP + FP) / (TP + TN + FP + FN)
print("Accuracy", accuracy)

precision = TP / (TP + FP)
print("Precision", precision)

recall = TP / (TP + FN)
print("Recall", recall)

f1_score = (2 * precision * recall) / (precision + recall)
print("F1-Score", f1_score)

mcc = ((TP * TN) - (FP * FN)) / np.sqrt((TP + FP) * (TP + FN) * (TN + FP) * (TN + FN))
print("MCC", mcc)


print(matrix_3x3)
print("For 3x3 matrix")

tp = matrix_3x3[0][0]
fn = matrix_3x3[0][1] + matrix_3x3[0][2]
fp = matrix_3x3[1][0] + matrix_3x3[2][0]
tn = matrix_3x3[1][1] + matrix_3x3[1][2] + matrix_3x3[2][1] + matrix_3x3[2][2]

accuracy = (tp + fp) / (tp + tn + fp + fn)
print("Accuracy", accuracy)
```

```
precision = tp / (tp + fp)
print("Precision", precision)

recall = tp / (tp + fn)
print("Recall", recall)

f1_score = (2 * precision * recall) / (precision + recall)
print("F1-Score", f1_score)

mcc = ((tp * tn) - (fp * fn)) / np.sqrt((tp + fp) * (tp + fn) * (tn + fp) * (tn + fn))
print("MCC", mcc)
```

Output

```
[[90 54]
 [22 42]]
For 2x2 Matrix
Accuracy 0.5384615384615384
Precision 0.8035714285714286
Recall 0.625
F1-Score 0.703125
MCC 0.2603869030610301
[[ 2 26 53]
 [ 7 71 88]
 [61 50 57]]
For 3x3 matrix
Accuracy 0.1686746987951807
Precision 0.02857142857142857
Recall 0.024691358024691357
F1-Score 0.026490066225165563
MCC -0.18935250340912718
```

Assignment 8

Question 1

Implement the following Morphological Operations using OpenCV in Python:

(a) Local Binary Pattern (LBP)
(b) Erosion
(c) Dilation
(d) Opening
(e) Closing

Solution

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("./gray_original.png", cv2.IMREAD_GRAYSCALE)

kernel = np.ones((5, 5), np.uint8)
erosion = cv2.erode(img, kernel, iterations=1)

dilation = cv2.morphologyEx(img, cv2.MORPH_DILATE, kernel)

opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)

closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)


def lbp(image):
    lbp_image = np.zeros_like(image)
    for i in range(1, image.shape[0] - 1):
        for j in range(1, image.shape[1] - 1):
            center = image[i, j]
            binary_string = ""
            binary_string += "1" if image[i - 1, j - 1] >= center else "0"
            binary_string += "1" if image[i - 1, j] >= center else "0"
            binary_string += "1" if image[i - 1, j + 1] >= center else "0"
            binary_string += "1" if image[i, j + 1] >= center else "0"
            binary_string += "1" if image[i + 1, j + 1] >= center else "0"
            binary_string += "1" if image[i + 1, j] >= center else "0"
            binary_string += "1" if image[i + 1, j - 1] >= center else "0"
            binary_string += "1" if image[i, j - 1] >= center else "0"
            lbp_value = int(binary_string, 2)
            lbp_image[i, j] = lbp_value
    return lbp_image


lbp_image = lbp(img)

plt.subplot(2, 3, 1)
_ = plt.imshow(img, cmap="gray")
_ = plt.title("Original Image")
_ = plt.axis("off")

plt.subplot(2, 3, 2)
_ = plt.imshow(erosion, cmap="gray")
```

```
_ = plt.title("Eroded Image")
_ = plt.axis("off")

plt.subplot(2, 3, 3)
_ = plt.imshow(dilation, cmap="gray")
_ = plt.title("Dilated Image")
_ = plt.axis("off")

plt.subplot(2, 3, 4)
_ = plt.imshow(lbp_image, cmap="gray")
_ = plt.title("LBP Image")
_ = plt.axis("off")

plt.subplot(2, 3, 5)
_ = plt.imshow(opening, cmap="gray")
_ = plt.title("Opened Image")
_ = plt.axis("off")

plt.subplot(2, 3, 6)
_ = plt.imshow(closing, cmap="gray")
_ = plt.title("Closed Image")
_ = plt.axis("off")

plt.tight_layout()
plt.show()
```

Output

```
<Figure size 640x480 with 6 Axes>
```